

Overflow-Detecting and Double-Wide Arithmetic Operations

Committee: ISO/IEC JTC1 SC22 WG21 SG6 Numerics

Document Number: P0103r1

Date: 2017-02-05

Authors: Lawrence Crowl

Reply To: Lawrence Crowl, Lawrence@Crowl.org

Abstract

We propose a set of functions for detecting overflow in basic integral operations and for basic integral operations producing double-wide results.

Contents

[Introduction](#)

[Problem](#)

[Solution](#)

[Overflow-Detecting Arithmetic](#)

[Double-Wide Arithmetic](#)

[Functions](#)

[Wording](#)

[???.1 Overflow-detecting arithmetic \[numbers.overarith\]](#)

[???.2 Double-wide arithmetic \[numbers.widearith\]](#)

[???.2.1 Typedefs \[numbers.widearith.typedefs\]](#)

[???.2.2 Composition functions \[numbers.widearith.composition\]](#)

[???.2.3 Wide-result functions \[numbers.widearith.wide\]](#)

[???.2.4 Split-result functions \[numbers.widearith.split\]](#)

[Revisions](#)

Introduction

Some integer arithmetic operations can overflow their representation. Machine architectures generally provide mechanisms to either detect or to construct larger representations in software.

- Machines commonly provide an overflow status, which can be tested in a branch instruction.
- Machines commonly provide a carry status, which can be used with an "add with carry" instruction. The "add with carry" instruction can help implement multi-word addition.
- Machines may provide a double-wide multiply instruction, which provides the upper and lower halves of a multiplication result in separate words. The double-wide multiply instruction can help implement multi-word multiplication.
- Machines may provide a double-wide divide instruction, which provides a dividend with double the size of the divisor. These can vary in the length of the quotient and whether they return a remainder. The double-wide division instruction can help implement multi-word division.
- Machines may provide a double-wide shift instructions, which provides a result double the size of the shifted value. The double-wide shift instructions can help implement scaling for fixed-point arithmetic.

Problem

C and C++ programmers have no standard mechanism to access the machine mechanisms. So, programmers coding within the standards resort to argument pre-checking or subword-emulation. Such code is difficult to get right and inefficient.

More perversely, because C, C++, and Fortran do not provide the mechanisms, machine architects are sometimes no longer including them, which makes some problems difficult to solve efficiently.

Getting compiler support for generating the appropriate instruction sequences is generally easier when the mechanisms span more than one language. This observation means that both C and C++ should support the same mechanism.

Solution

We propose a set of functions that provide access to machine mechanisms. We do *not* intend that these functions be end-user functions. Our intent is that these functions be tools for library authors to implement types more appropriate to end users. The proposal here is essentially a hardware abstraction layer.

With use limited to library authors, the syntax can be somewhat more demanding, which in turn means that a syntax suitable for both C and C++ is acceptable. We rely on type-generic macros in C and overloading in C++. Note however, that the operations are applicable beyond built-in types in C++. For larger types, const reference parameters may be more appropriate.

Overflow-Detecting Arithmetic

The overflow-detecting functions return a boolean true when the operation overflows, and a boolean false when the operation does not overflow. Compilers may assume that a true result is rare. When the return is false, the function writes the operation result through the given pointer. When the return is true, the pointer is not used and no write occurs.

Double-Wide Arithmetic

There are two classes of functions, those that provide a result in a double-wide type and those that provide a result split into two single-wide types. The latter serves well in implementing multi-word arithmetic.

double-wide result

The result is the function return value.

split result

The high-order half of the result is the function return value. In multi-word arithmetic, this half is the carry. The low-order half of the result written through an argument pointer. In multi-word arithmetic, this half is the portion to be stored in a word. To facilitate multi-word arithmetic, types of the halves are always unsigned. The intent is that in loops, the lower part is written once to memory while the upper part is carried between iterations in a local variable.

When implementing multi-word arithmetic, the appropriate word size depends on the target architecture. To provide the appropriate word size, we provide a set of typedefs for the natural/efficient size of signed and unsigned versions of both the single-wide and double-wide types.

We provide functions to build a double-wide type from two halves and to split a double-wide type into halves.

Functions

We use the following codes are components of function names.

code	meaning
cv t	The value converted to the result type.
neg	The negative of the value.
add	The sum of the augend and addend.
sub	The difference of minuend and subtrahend.
add2	The sum of the augend and two addends. This operation is useful in multiword addition.
sub2	The difference of the minuend and two subtrahends. This operation is useful in multiword subtraction.
lsh	The multiplicand shifted left by the count, i.e. the product of the multiplicand and 2 ^{count} .
lshadd	The sum of the multiplicand shifted left by the count and the addend. i.e. the sum of (the product of the multiplicand and 2 ^{count}) and the addend. The first value shifted left by the count and then sumed with the second value. This operation is useful with multiword scaled addition.
mul	The product of the multiplier and multiplicand.
muladd	The sum of (the product of the multiplicand and the multiplier) and the addend. This operation is useful with multiword multiplication.
muladd2	The sum of (the product of the multiplicand and the multiplier) and the two addends. This operation is useful with multiword multiplication.
mulsub	The difference of (the product of the multiplicand and the multiplier) and the subtrahend.

mulsub2	The difference of (the product of the multiplicand and the multiplier) and the two subtrahend.
divn	The narrow quotient of the dividend and divisor.
divw	The wide quotient of the dividend and divisor.
divnrem	As with divn except also computing a remainder with the sign of the divisor.
divwrem	As with divw except also computing a remainder with the sign of the divisor.

Wording

?.?.1 Overflow-detecting arithmetic [numbers.overarith]

Add a new section:

```
template <typename C, typename T> bool overflow_cvt( C* result, T a );  
template <typename T> bool overflow_neg( T* result, T a );  
template <typename T> bool overflow_lsh( T* result, T a, int b );  
template <typename T> bool overflow_add( T* result, T a, T b );  
template <typename T> bool overflow_sub( T* result, T a, T b );  
template <typename T> bool overflow_mul( T* result, T a, T b );  
template <typename T> bool overflow_div( T* result, T a, T b );
```

Effects:

Computes an expression as in the following table.

overflow_add	a+b	overflow_mul	a*b	overflow_neg	-a
overflow_sub	a-b	overflow_div	a/b	overflow_lsh	a<<b
				overflow_cvt	static_cast<C>(a)

If there is no overflow, writes the computed value to *result.

Returns:

true if the computation overflows, false otherwise.

Notes:

- Compilers may assume that overflow is rare.
- Overflow can occur for the expressions -MIN_INT and MIN_INT/-1 with two's complement representation.

?.?.2 Double-wide arithmetic [numbers.widearith]

Add a new section:

In the specifications below.

D(T)

identifies a type with the same attributes as T except with double the size.

H(T)

identifies a type with the same attributes as T except with half the size.

S(T)

identifies a type with the same attributes as T except that it is signed.

U(T)

identifies a type with the same attributes as T except that it is unsigned.

???.2.1 Typedefs [numbers.widearith.typedefs]

Add a new section:

```
typedef implementation-defined single_sword;  
typedef U(single_sword) single_uword;  
typedef D(single_sword) double_sword;  
typedef D(single_uword) double_uword;
```

Requires:

single_sword is a signed integral type.

???.2.2 Composition functions [numbers.widearith.composition]

Add a new section:

```
template <typename T> H(U(T)) split_upper( T a );  
template <typename T> H(U(T)) split_lower( T a );
```

Requires:

T is integral.

Returns:

for split_upper, the high-magnitude half of a; for split_lower, the low-magnitude half of a.

```
template <typename T> S(D(T)) wide_signed( T a, T b );  
template <typename T> U(D(T)) wide_unsigned( T a, T b );
```

Requires:

T is integral and unsigned.

Returns:

the value formed by concatenating the two halves, with a high and b low.

???.2.3 Wide-result functions [numbers.widearith.wide]

Add a new section:

```
template <typename T> D(T) wide_neg( T a );  
template <typename T> D(T) wide_add( T a, T b );  
template <typename T> D(T) wide_sub( T a, T b );  
template <typename T> D(T) wide_add2( T a, T b, T c );  
template <typename T> D(T) wide_sub2( T a, T b, T c );  
template <typename T> D(T) wide_lsh( T a, int b );  
template <typename T> D(T) wide_lshadd( T a, int b, T c );  
template <typename T> D(T) wide_lshsub( T a, int b, T c );  
template <typename T> D(T) wide_mul( T a, T b );  
template <typename T> D(T) wide_muladd( T a, T b, T c );  
template <typename T> D(T) wide_mulsub( T a, T b, T c );  
template <typename T> D(T) wide_muladd2( T a, T b, T c, T d );  
template <typename T> D(T) wide_mulsub2( T a, T b, T c, T d );  
template <typename T> T wide_divn( D(T) a, T b );  
template <typename T> D(T) wide_divw( D(T) a, T b );
```

Requires:

- τ is integral.
- For `wide_lsh`, `wide_lshadd` and `wide_lshsub`, the shift count `b` shall be non-negative and less than the number of bits in τ .
- For `wide_muladd` and `wide_mulsub`, at least one of `a` and `b` shall not be the minimum integer in two's complement representation.
- For `wide_muladd2` and `wide_mulsub2`, both of `a` and `b` shall not be the minimum integer in two's complement representation.
- For `wide_divn` and `wide_divw`, the divisor (`b`) shall not be zero.

Effects:

Computes an expression as in the following table, with each argument (except shift count) promoted to $D(\tau)$.

<code>wide_neg</code>	<code>-a</code>	<code>wide_lsh</code>	<code>a<<b</code>	<code>wide_mul</code>	<code>a*b</code>
<code>wide_add</code>	<code>a+b</code>	<code>wide_lshadd</code>	<code>(a<<b)+c</code>	<code>wide_muladd</code>	<code>a*b+c</code>
<code>wide_sub</code>	<code>a-b</code>	<code>wide_lshsub</code>	<code>(a<<b)-c</code>	<code>wide_mulsub</code>	<code>a*b-c</code>
<code>wide_add2</code>	<code>a+b+c</code>	<code>wide_divn</code>	<code>a/b</code>	<code>wide_muladd2</code>	<code>a*b+c+d</code>
<code>wide_sub2</code>	<code>a-b-c</code>	<code>wide_divw</code>	<code>a/b</code>	<code>wide_mulsub2</code>	<code>a*b-c-d</code>

Returns:

The computed value.

```
template <typename T> T wide_divnrem( T* remainder, D(T) a, T b );
template <typename T> D(T) wide_divwrem( T* remainder, D(T) a, T b );
```

Requires:

- τ is integral.
- The value of the expression `a/b` shall be representable in the return type.
- The divisor (`b`) shall not be zero.

Effects:

Computes a quotient (`a/b`) and a remainder (`a%b`). The remainder shall have the same sign as the divisor (`b`).
Writes the remainder to `*remainder`.

Returns:

The quotient.

??2.4 Split-result functions [numbers.widearith.split]

Add a new section:

```
template <typename T> U(T) split_neg( U(T)* low, T a );
template <typename T> U(T) split_add( U(T)* low, T a, T b );
template <typename T> U(T) split_sub( U(T)* low, T a, T b );
template <typename T> U(T) split_add2( U(T)* low, T a, T b, T c );
template <typename T> U(T) split_sub2( U(T)* low, T a, T b, T c );
template <typename T> U(T) split_lsh( U(T)* low, T a, int b );
template <typename T> U(T) split_lshadd( U(T)* low, T a, int b, T c );
template <typename T> U(T) split_lshsub( U(T)* low, T a, int b, T c );
template <typename T> U(T) split_mul( U(T)* low, T a, T b );
template <typename T> U(T) split_muladd( U(T)* low, T a, T b, T c );
template <typename T> U(T) split_mulsub( U(T)* low, T a, T b, T c );
template <typename T> U(T) split_muladd2( U(T)* low, T a, T b, T c, T d );
template <typename T> U(T) split_mulsub2( U(T)* low, T a, T b, T c, T d );
```

```
template <typename T> T split_divn( U(T)* low, U(T) ah, U(T) al, T b );
template <typename T> U(T) split_divw( U(T)* low, U(T) ah, U(T) al, T b );
```

Requires:

- T is integral.
- For split_lsh, split_lshadd and split_lshsub, the shift count b shall be non-negative and less than the number of bits in T.
- For split_muladd and split_mulsub, at least one of a and b shall not be the minimum integer in two's complement representation.
- For split_muladd2 and split_mulsub2, both of a and b shall not be the minimum integer in two's complement representation.
- For split_divn and split_divw, the divisor (b) shall not be zero.

Effects:

Computes a value (a) of type D(T) by concatenating the high (ah) and low (al) halves. Computes an expression as in the following table, with each argument (except shift count) promoted to D(T).

split_neg	-a	split_lsh	a<<b	split_mul	a*b
split_add	a+b	split_lshadd	(a<<b)+c	split_muladd	a*b+c
split_sub	a-b	split_lshsub	(a<<b)-c	split_mulsub	a*b-c
split_add2	a+b+c	split_divn	a/b	split_muladd2	a*b+c+d
split_sub2	a-b-c	split_divw	a/b	split_mulsub2	a*b-c-d

Writes the low half of the computed result to *low.

Returns:

The high half of the computed result.

```
template <typename T> T split_divnrem( T* remainder, U(T) ah, U(T) al, T b );
template <typename T> U(T) split_divwrem( U(T)* low, T* remainder, U(T) ah, U(T) al, T b );
```

Requires:

- T is integral.
- For split_divnrem, the value of the expression a/b shall be representable in T. For split_divwrem, the value of the expression a/b shall be representable in D(T).
- The divisor (b) shall not be zero.

Effects:

Computes a value (a) of type D(T) by concatenating the high (ah) and low (al) halves. Computes a quotient (a/b) and a remainder (a%b). The remainder shall have the same sign as the divisor (b). Writes the remainder to *remainder. For split_divwrem, writes the low half of the computed result to *low.

Returns:

For split_divnrem, the quotient; for split_divwrem, the high half of the quotient.

Revisions

This paper modifies [P0103R0](#) - 2015-09-27.

- Add Revisions section.
- Add Wording section.
- Move non-explanatory definitional statements from the Solution to the Wording.

- Remove the reference to [P0102R0 C++ Parametric Number Type Aliases](#).
- Remove the type macros. A set of typedefs for the natural size is sufficient.
- Add `overflow_div` to handle the case of two's complement `MIN_INT/-1`.
- Add `wide_neg` and `split_neg` to handle the case of two's complement `-MIN_INT`.
- Make the decomposed halves of a double-wide number both be unsigned. This change aids implementation of multi-word numbers as arrays.
- Remove Open Issues section.